

Predictive AI-IoT Framework for Early Post-Harvest Spoilage Detection

1. Complete Components List

- **Arduino Uno R3:** Main microcontroller board (Quantity: 1)
- **MQ-3 Gas Sensor:** Detects ethanol and VOC emissions (Quantity: 1)
- **DHT11 Sensor:** Measures ambient temperature and humidity (Quantity: 1)
- **Red LED:** Visual spoilage alert indicator (Quantity: 1)
- **Green LED:** Visual healthy baseline indicator (Quantity: 1)
- **Active Piezo Buzzer:** Audible local spoilage alarm (Quantity: 1)
- **220Ω Resistors:** Current limiters for LEDs to prevent burnout (Quantity: 2)
- **Half-size Breadboard:** Base for prototyping without soldering (Quantity: 1)
- **Jumper Wires:** Male-to-Male and Male-to-Female for connections (Quantity: 15)

2. Circuit Wiring Guide

Power Distribution

- Connect the **5V pin** on the Arduino to the positive (+) rail of the breadboard.
- Connect the **GND pin** on the Arduino to the negative (-) rail of the breadboard.

Sensor Nodes

- **MQ-3 Gas Sensor:** Connect VCC to (+), GND to (-), and AOUT to **Arduino Analog Pin A0**.
- **DHT11 Sensor:** Connect VCC to (+), GND to (-), and DATA to **Arduino Digital Pin 2**.

Actuator Nodes (Backup Alert System)

- **Green LED:** Connect anode (long leg) to **Arduino D6**. Connect cathode (short leg) to a 220Ω resistor, then to (-).
- **Red LED:** Connect anode to **Arduino D7**. Connect cathode to a 220Ω resistor, then to (-).
- **Piezo Buzzer:** Connect positive pin to **Arduino D8**. Connect negative pin to (-).

3. Final Arduino Code (Hardware Node)

```
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

const int mq3Pin = A0;
const int greenLed = 6;
const int redLed = 7;
const int buzzerPin = 8;

// Hardware baseline: Alerts if local system disconnects from AI
const int gasThreshold = 300;

void setup() {
  Serial.begin(9600);
  dht.begin();

  pinMode(greenLed, OUTPUT);
  pinMode(redLed, OUTPUT);
  pinMode(buzzerPin, OUTPUT);
}

void loop() {
  float hum = dht.readHumidity();
  float temp = dht.readTemperature();
  int gasVal = analogRead(mq3Pin);

  if (isnan(hum) || isnan(temp)) {
    Serial.println("Error reading DHT");
    delay(2000);
    return;
  }

  // Local Offline Backup Alarm
  if (gasVal > gasThreshold) {
```

```

        digitalWrite(redLed, HIGH);
        digitalWrite(greenLed, LOW);
        digitalWrite(buzzerPin, HIGH);
    } else {
        digitalWrite(redLed, LOW);
        digitalWrite(greenLed, HIGH);
        digitalWrite(buzzerPin, LOW);
    }

    // Stream clean CSV string to Python ML pipeline
    Serial.print(temp);
    Serial.print(",");
    Serial.print(hum);
    Serial.print(",");
    Serial.println(gasVal);

    // 2-second delay required for DHT11 stability
    delay(2000);
}

```

4. Synthetic Data Generation & AI Training (train_model.py)

```

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
import joblib

print("Generating realistic agricultural sensor data...")

np.random.seed(42)
n_samples = 3000

# Generate realistic environmental profiles (India/Ahmedabad conditions)
temp = np.random.normal(28, 5, n_samples)
hum = np.random.normal(65, 15, n_samples)
gas = np.random.normal(200, 100, n_samples)

temp = np.clip(temp, 15, 45)
hum = np.clip(hum, 30, 95)
gas = np.clip(gas, 50, 800)

risk = np.zeros(n_samples)

for i in range(n_samples):

```

```

# Real-world logic: Risk primarily spikes when gas crosses ~220 ppm
gas_factor = max(0, (gas[i] - 220) / 4)

# Synergistic effect: High temp/humidity multiply the rotting speed
temp_multiplier = 1.0 + max(0, (temp[i] - 30) * 0.05)
hum_multiplier = 1.0 + max(0, (hum[i] - 70) * 0.02)

calculated_risk = gas_factor * temp_multiplier * hum_multiplier

# Add real-world sensor noise/jitter to prevent model overfitting
noise = np.random.normal(0, 4)
risk[i] = calculated_risk + noise

# Bound the output rigidly between 0% and 100%
risk = np.clip(risk, 0.0, 100.0)

X = pd.DataFrame({'Temp': temp, 'Humidity': hum, 'Gas': gas})
y = risk

print("Training non-linear Random Forest AI Model...")
model = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42)
model.fit(X, y)

joblib.dump(model, 'spoilage_model.pkl')
print("Model saved successfully as 'spoilage_model.pkl'. Ready for deployment.")

```

5. Main Dashboard Pipeline (dashboard.py)

```

import serial
import time
import tkinter as tk
import joblib
import pandas as pd

# 1. Load the trained AI model
try:
    ai_model = joblib.load('spoilage_model.pkl')
    print("AI Model loaded successfully!")
except:
    print("Error: Run train_model.py first to generate the AI model.")
    exit()

# 2. Configure Hardware Connection (UPDATE THIS PORT)
SERIAL_PORT = 'COM3'

```

```
BAUD_RATE = 9600
```

```
try:
    arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)
except Exception as e:
    print(f"Error opening serial port: {e}")
    exit()
```

```
# 3. Moving Average Buffer
```

```
gas_history = []
```

```
def update_dashboard():
```

```
    try:
        if arduino.in_waiting > 0:
            line = arduino.readline().decode('utf-8').strip()
            data = line.split(',')

            if len(data) == 3:
                temp = float(data[0])
                hum = float(data[1])
                gas = int(data[2])

                # Smooth out sudden false spikes
                gas_history.append(gas)
                if len(gas_history) > 5:
                    gas_history.pop(0)
                avg_gas = sum(gas_history) / len(gas_history)

                # Execute live prediction
                live_data = pd.DataFrame({'Temp': [temp], 'Humidity': [hum],
'Gas': [avg_gas]})
                risk_percentage = ai_model.predict(live_data)[0]

                # Update UI Elements
                temp_label.config(text=f"Temperature: {temp} °C")
                hum_label.config(text=f"Humidity: {hum} %")
                gas_label.config(text=f"VOC Level: {int(avg_gas)} ppm")
                risk_label.config(text=f"Spoilage Risk: {risk_percentage:.1f}%")

                # Dynamic Visual Alerts
                if risk_percentage > 75.0:
                    status_label.config(text="ACTION REQUIRED: High Spoilage
Risk!", fg="red")
                    root.configure(bg="#ffebee")
                elif risk_percentage > 40.0:
                    status_label.config(text="WARNING: Early Stage Spoilage",
```

```

fg="orange")
        root.configure(bg="#fff8e1")
    else:
        status_label.config(text="STATUS: Batch is Healthy",
fg="green")
        root.configure(bg="#f4f4f4")

    except Exception as e:
        print(f"Data error: {e}")

    root.after(500, update_dashboard)

# 4. GUI Layout Initialization
root = tk.Tk()
root.title("Predictive AI Spoilage Dashboard")
root.geometry("500x400")
root.configure(bg="#f4f4f4")

tk.Label(root, text="Godown Sensor Node 1", font=("Arial", 18, "bold"),
bg="#f4f4f4").pack(pady=15)
temp_label = tk.Label(root, text="Temperature: -- °C", font=("Arial", 14),
bg="#f4f4f4")
temp_label.pack(pady=5)
hum_label = tk.Label(root, text="Humidity: -- %", font=("Arial", 14),
bg="#f4f4f4")
hum_label.pack(pady=5)
gas_label = tk.Label(root, text="VOC Level: -- ppm", font=("Arial", 14),
bg="#f4f4f4")
gas_label.pack(pady=5)
risk_label = tk.Label(root, text="Spoilage Risk: --%", font=("Arial", 22,
"bold"), fg="blue", bg="#f4f4f4")
risk_label.pack(pady=20)
status_label = tk.Label(root, text="Connecting to Sensors...", font=("Arial",
14, "bold"), bg="#f4f4f4")
status_label.pack(pady=10)

root.after(1000, update_dashboard)
root.mainloop()

```

6. Exact Execution Strategy

Phase 1: Assembly & Pre-Check

- Assemble the hardware components on the breadboard exactly as detailed in the wiring guide.
- Plug the Arduino into your computer. Open the Arduino IDE, verify your specific port (e.g., COM3), paste the Arduino C++ code, and click **Upload**.
- Briefly open the Arduino IDE Serial Monitor to confirm data is printing (e.g., 32.00,68.00,145).
- **Close the Serial Monitor.** (Leaving it open will block Python from reading the sensor).

Phase 2: Environment Setup & AI Training

- Create a new, empty folder on your desktop named `NCSC_Project`.
- Open a terminal/command prompt inside that folder and run: `pip install scikit-learn pandas pyserial`.
- Create a file named `train_model.py` in that folder, paste the synthetic generation script, and run it via terminal: `python train_model.py`.
- Verify that `spoilage_model.pkl` now exists in the folder. You never need to run this script again.

Phase 3: Live System Launch & Demonstration

- Create `dashboard.py` in the same folder. Paste the dashboard code into it. Ensure line 17 matches your Arduino port.
- Place the physical Arduino setup inside a small, clear plastic storage container with a fresh piece of fruit.
- Run the dashboard script via terminal: `python dashboard.py`.
- Allow the system to stabilize. The screen will show normal environmental data and a low spoilage risk (< 10%).
- **The Demonstration:** During your presentation, wave a tissue slightly dampened with alcohol-based hand sanitizer near the sensor inside the box. The MQ-3 will immediately read the volatile ethanol vapors. The screen will instantly spike the VOC numbers, the AI model will recalculate the risk to over 80%, the laptop screen will turn red, and the physical buzzer on the breadboard will sound.